

Name: Atul Kumar	Reg.No: 12102801
Roll No: RD2110B61	CA: 02 Set: "5"
Course Code: CAP-615	Instructor: Mr. Kr. Vishal
Question No: 01	Total Page: 10

Q1 How would you describe Interthread communication? How you will use it to solve Producer consumer problem using a suitable program?

Ans: Interthread communication in java is all about allowing synchronized threads to communicate with each other. Interthread communication is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter in the same critical section to be executed. For example, suppose that there are two threads A and B. Thread B uses data produced by Thread A and perform its task. If Thread B waits for thread A to produce data, it will waste many CPU cycles. But if Thread A & B communicate with each other when they have completed their tasks, they do not have to wait and check each other's status.

Page: 01 of 10

Name: Atul Kumar	Reg. No: 12102801
Roll No: RD2110B61	Course Code: CAP615
Question No: 01	Assignment: 02

Thus, CPU cycles will not waste. This type of information exchanging between thread is called interthread communication.

For achieving inter-thread communication

There are three methods which are used to achieve inter-thread communication:-

- i) wait() method: It causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object.
- ii) notify() method: It wakes up a single thread that is waiting on this object's monitor. If any thread are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.
- iii) notifyAll() method: It wakes up all the threads that are waiting on this object's monitor.

Name: Atul Kumar

Reg. No: 12102801

Roll No: RD2110B61

Course code: CAP-615

Question No: 01

Assignment: 02

Producer consumer Problem

```
import java.lang.*;
```

```
class ProducerConsumer {
```

```
    public static void main (String[] args) {
```

```
        Shop c = new Shop();
```

```
        Producer p1 = new Producer(c, 1);
```

```
        Consumer c1 = new Consumer(c, 1);
```

```
        p1.start();
```

```
        c1.start();
```

```
    }
```

```
}
```

```
class Shop
```

```
{
```

```
    private int materials;
```

```
    private boolean available = false;
```

```
    public synchronized int get()
```

```
    { while (available == false) {
```

```
        try {
```

```
            wait();
```

```
        }
```

```
        catch (InterruptedException ie)
```

```
        {}}
```

```
        available = false;
```

```
        notifyAll();
```

```
        return materials;
```

```
    }
```

Page: 03 of 10

Name: Atul Kumar

Reg. No: 12102801

Roll No: RD2110B61 Course Code: CAP-615

Question No: 01

Assignment: 02

```
public synchronized void put (int value)
```

```
{ while (available == true)
```

```
{ try
```

```
{ wait();
```

```
}
```

```
catch (InterruptedException ie)
```

```
{
```

```
ie.printStackTrace();
```

```
}
```

```
material = value;
```

```
available = true;
```

```
notifyAll();
```

```
}
```

```
class Consumer extends Thread {
```

```
private Shop shop;
```

```
private int number;
```

```
public Consumer(Shop c, int number)
```

```
{ shop = c;
```

```
this.number = number;
```

```
}
```

```
public void run()
```

```
{ int value = 0;
```

```
for (int i = 0; i < 5; i++)
```

```
{
```

```
value = shop.get();
```

```
System.out.println("Consumed value " + this.number
```

```
+ " got: " + value);
```

```
}
```

```
}
```

```
}
```

Page 04 of 10

class Producer extends Thread

```

{
    private Shop shop;
    private int number;
    public Producer(Shop c, int number)
    {
        shop = c;
        this.number = number;
    }
    public void run()
    {
        for (int i = 0; i < 5; i++)
        {
            shop.put(i);
            System.out.println("Produced value " + this.number + " put: " + i);
            try {
                sleep((int)(Math.random()*100));
            }
            catch (InterruptedException ie)
            {
                i.e..printStackTrace();
            }
        }
    }
}

```

Page: 05 of 10

C:\javaprograms>javac ProducerConsumer.java

C:\javaprograms>java ProducerConsumer

Produced value 1 put: 0

Consumed value 1 got: 0

Produced value 1 put: 1

Consumed value 1 got: 1

Consumed value 1 got: 2

Produced value 1 put: 2

Produced value 1 put: 3

Consumed value 1 got: 3

Produced value 1 put: 4

Consumed value 1 got: 4

Name: Atul Kumar

Reg. No: 12102801

Roll No: RD2110B61

Course Code: CAP-615

Question No: 02

Assignment: 02

Q2. "Each Thread may perform different tasks. Sometimes, it becomes necessary to suspend the execution of a thread for a period of time". Justify your answer.

Ans: Each thread perform different tasks but sometimes it becomes necessary to temporarily suspend the execution of a thread by making it go through from running state to waiting for the state. The concept used in achieving the goal is the `suspend()` function.

How the thread can be suspended?

- i) Creating 'MyThread Class' which is extending the 'java.lang.Thread class'. It has a `run()` function which contains some code to perform.
- ii) In the main function, a 'MyThread class' object is created 'thread' which is named as 'SuspendThrExe' by using the `setName()` function.

Page: 06 of 10

Name: Atul Kumar

Reg. No: 12102801

Roll No: RD2110B81

Course Code: CAP-615

Question No: 02

Assignment: 02

iii) Start the thread to perform its tasks by calling the `start()` function and it starts executing the code written in the `run()` function.

Methods Used:

i) `start()` ii) `setName` iii) `sleep` iv) `suspend` v) `resume`

Example:

```
import java.util.*;
```

```
class MyThread extends Thread {
```

```
    public void run()
```

```
    {
```

```
        try {
```

```
            System.out.println("Thread " + Thread.currentThread().getName() + " is running");
```

```
        }
```

```
        catch (Exception e) {
```

```
            System.out.println("Exception is caught");
```

```
        }
```

```
    }
```

```
}
```

```
public class SuspendThread {
```

```
    public static void main (String[] args) throws Exception
```

```
    {
```

Page: 07 of 10

Name: Atul Kumar

Reg. No. 12102801

Roll No: RD2110B61

Course Code: CAP-615

Question No: 02

Assignment: 02

```
MyThread thread = new MyThread();  
thread.setName("suspendThrExe");  
thread.start();  
thread.sleep(500);  
thread.suspend();  
System.out.println("Thread going to  
sleep for 5 seconds.");  
thread.sleep(5000);  
System.out.println("Thread Resumed");  
thread.resume();  
}  
}
```

Output

Thread 10 is running
Thread Suspendded (Nothing could print)
Thread going to sleep for 5 seconds
Thread Resumed

Page: 08 of 10

Name: Atul Kumar
Roll No: RD2110BB1
Question No: 03

Reg. No: 12102801
Course Code: CAP-615
Assignment: 02

Q3. How would you define Event? Give example of events. Define event handler and how it handles events?

Ans: Event: An event is a signal received by a program from the operating system as a result of some action taken by the user or because something else has happened. Events are generated as result of user interaction with the GUI component
For example

- i) Clicking on a mouse.
- ii) Moving the mouse.
- iii) Entering some data from keyboard.
- iv) Scrolling the page.
- v) Closing a window

Event Handling: Event handling is fundamental to Java programming because it is used to create event driven programs i.e. Applets, GUI based window application, web application etc. It is a mechanism that controls the event and decides what should happen if an event occurs. This mechanism has the code which is known as event handler that is executed when an event occurs.

Page: 09 of 10

Name: Atul Kumar

Reg. No: 12102801

Roll No: RD2110B61

Course Code: CAP-615

Question No: 03

Assignment: 02

Java uses the delegation event model to handle the events. This model defines the standard mechanism to generate and handle the events.

The Delegation event model has following key participants:-

- a) Source - The source is an object on which event occurs. It is responsible for information.
- b) Listener - It is also known as event handler. Listener is responsible for generating response to an event.

Steps involved in event handling

- i) The user clicks the button & the event generated
- ii) Now the object of concerned event class is created automatically & information about the source and the event get populated with in same object.
- iii) Event object is forwarded to the method of registered listener class.
- iv) The method is now get executed and returns.

Page: 10 of 10



thank you!